

Backtracking Problems

~~~~~

## Agenda

~~~~~

1. Subset sum equals to k
2. Print paths in staircase problem
3. Print all paths from source to destination
4. Shortest path in a matrix with huddles

Hello Everyone
very special Good Morning
to All of you. 😊
We will start
from 11:06 AM

~~~~~ Subset sum equals to k

~~~~~  
Given an integer array A of size N and an integer B, you need to find whether there exist a subset in A whose sum equal B.

arr: [ 3, 34, -3, 12, 5, 2 ] , B = 9  
      0 1 2 3 4 5

$-10^9 \leq \text{arr}[i] \leq 10^9$

output: true

arr: [ -8, 34, 4, 0, -5, -2 ] , B = -20  
      0 1 2 3 4 5

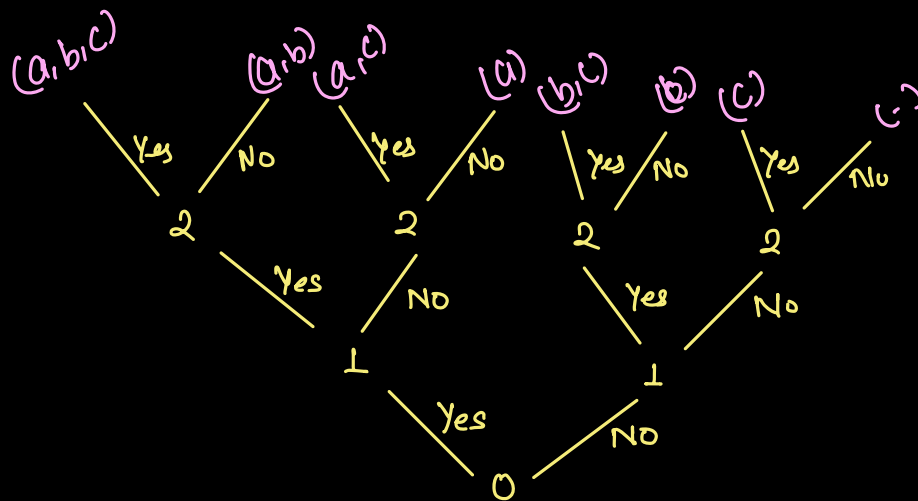
output: false

Quiz 1: [-5, 9, -1, 6]

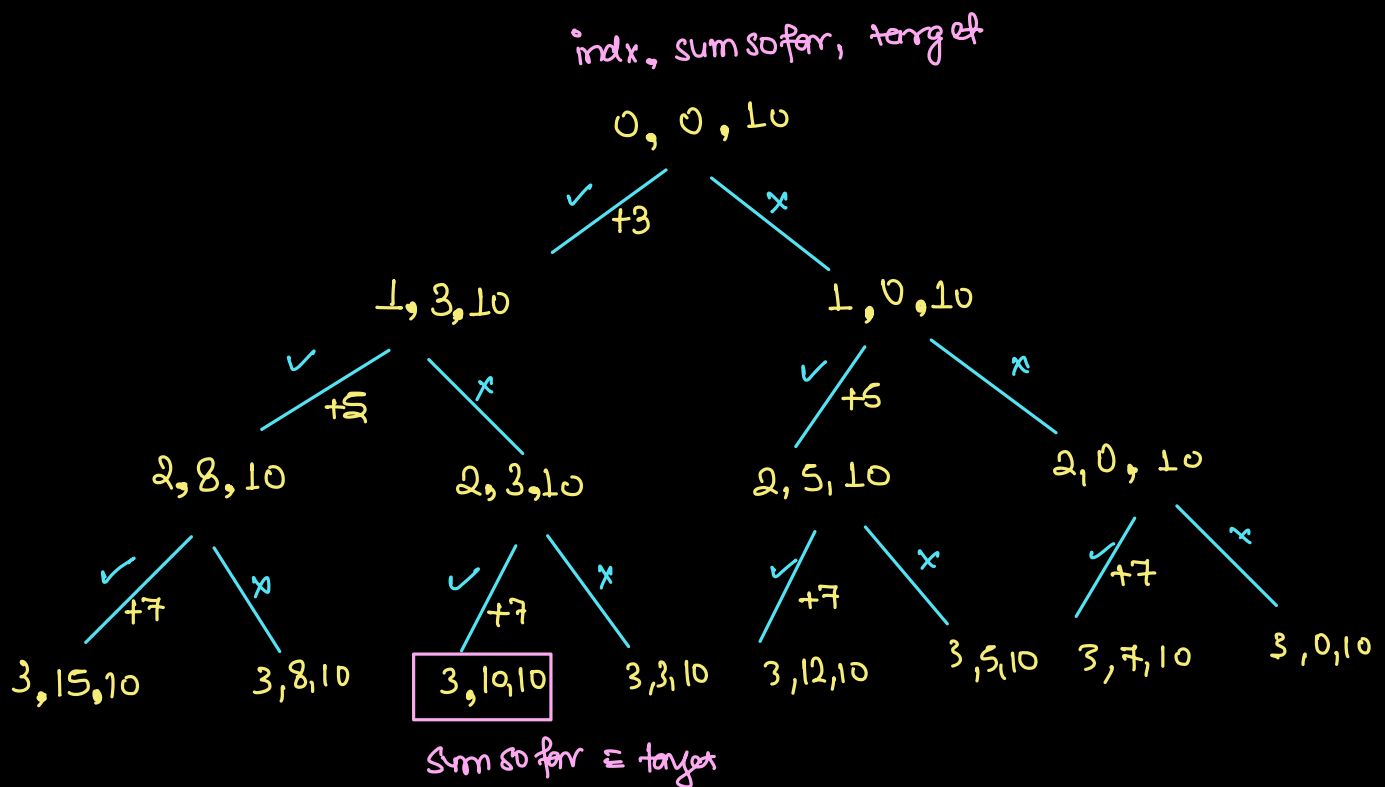
subset  $\rightarrow \{-5, 9, 6\} \rightarrow \text{sum} = 10$

$\{a, b, c\} \rightarrow$  generate all subset  
 0 1 2

subarray  
 $\rightarrow$  continuity



arr: [3, 5, 7], B = 10



```
boolean findSubset(int[] arr, int B) {
```

ssf → sum so far

```
    return solve(arr, 0, 0, B);
}
```

```
boolean solve(int[] arr, int indx, int ssf, int target) {
```

```
    if (indx == length of array) {
```

```
        return ssf == target;
```

```
    }
```

```
    // yes call
```

```
    boolean res = solve(arr, indx+1, ssf + arr[indx], target);
```

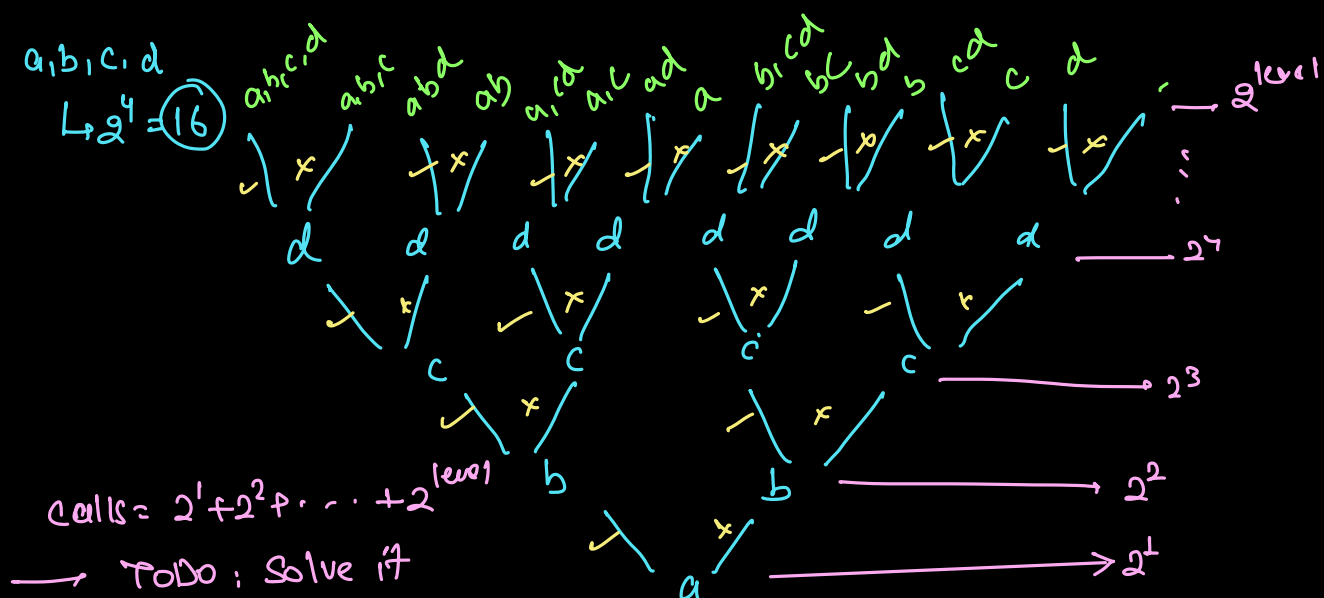
```
    // No call
```

```
    res = res || solve(arr, indx+1, ssf, target);
```

```
    return res;
```

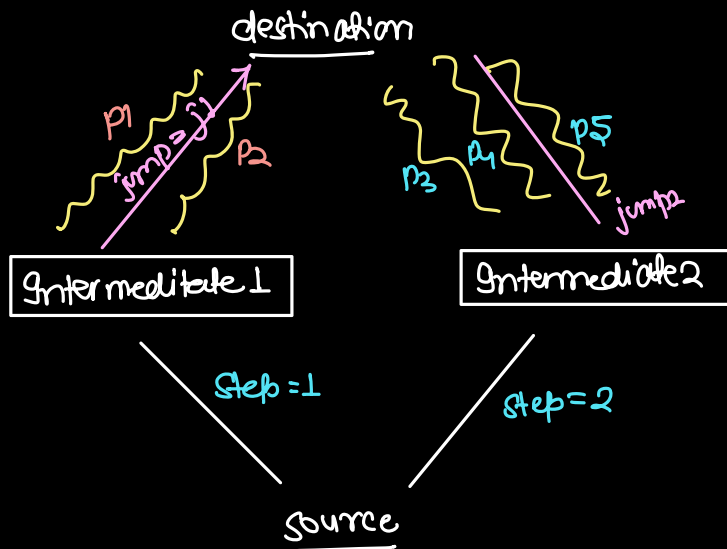
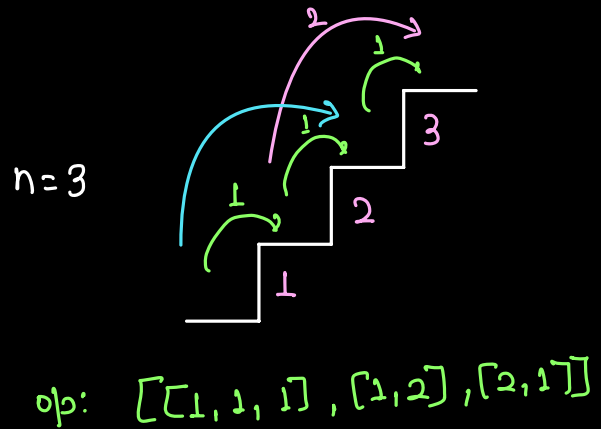
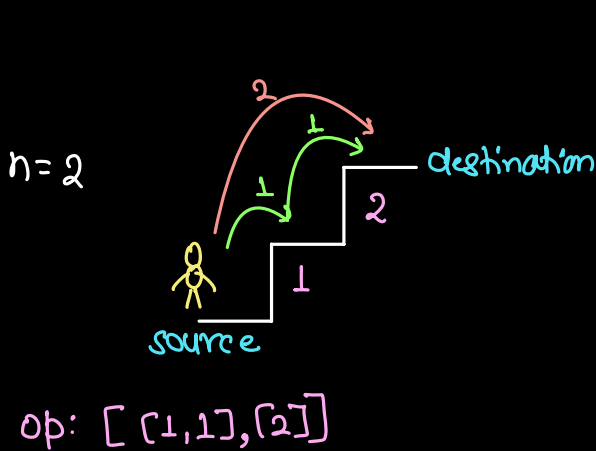
```
}
```

T.C:  $O(2^n)$  Why: Explore T.C of subset problem  
S.C:  $O(n)$  → creating n levels in recursive stack. so it is.



# Print paths in staircase problem

You are climbing a staircase and it takes A steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top? You need to return all the distinct ways to climb to the top in lexicographical order.



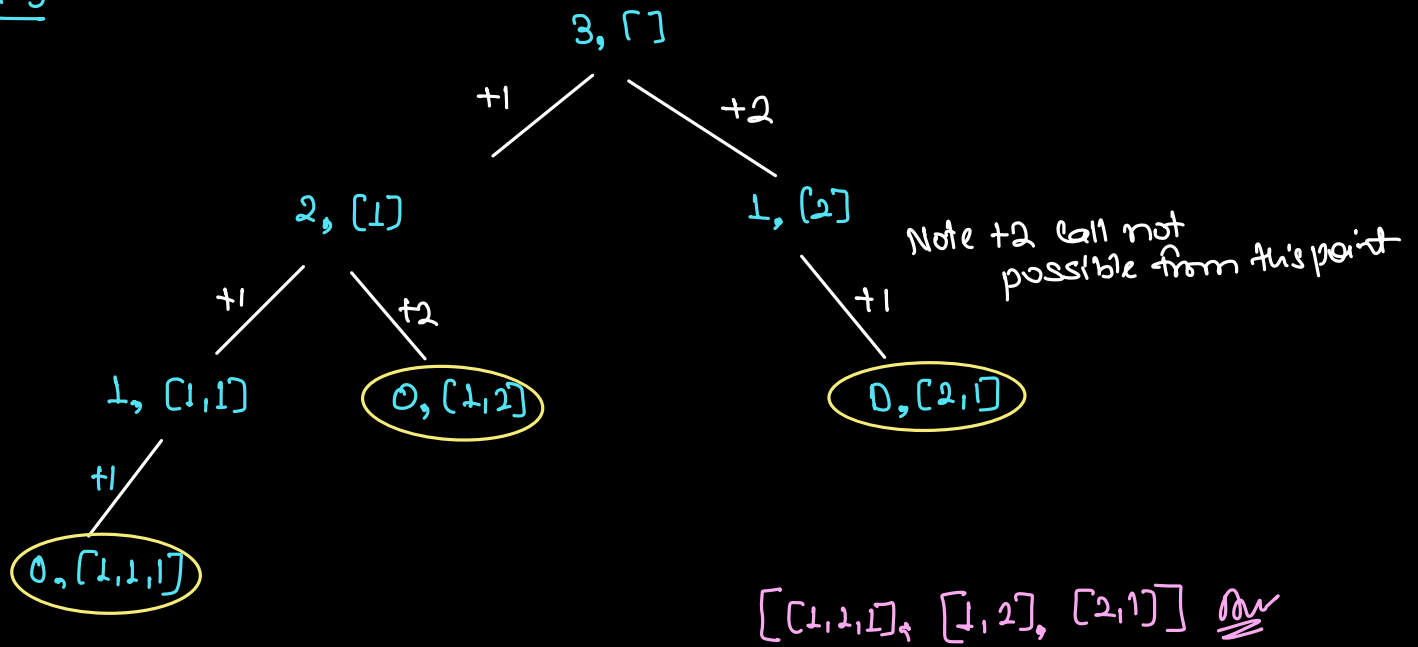
path  $\rightarrow$

Sum of path from all possible intermediate points.

(min jumps)  
 Jump  $\rightarrow \min(j_1, j_2) + 1$   
 min of all intermediate jumps + 1



n=3



2D-Dynamic array  $\rightarrow$  ans

void generatePath (int stairs, int[] dynamic Array) {

if (stairs == 0) {

1. make a copy of dynamic array.
2. Add that copy in global ans.

ans.add(copy);

return;

}

if (stairs >= 1) {

dynamic Array.add(1);

generatePath (stairs - 1, dynamic Array);

dynamic Array.removeLast();

}

if (stairs >= 2) {

dynamic Array.add(2);

generatePath (stairs - 2, dynamic Array);

dynamic Array.removeLast();

}

T.C:  $O(2^n)$

S.C:  $O(n)$

TODO:

$\rightarrow$  Try to relate this problem with 'fibonacci'

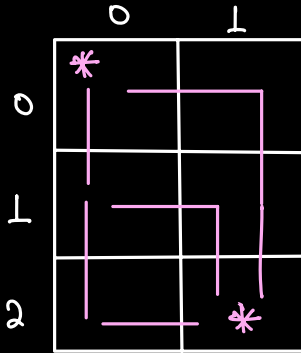
$\rightarrow$  Solve total no. of paths.

12:15 - 12:25 pm

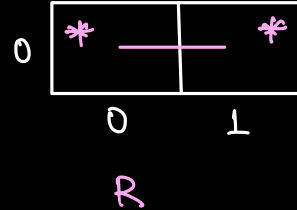
~~~~~  
Print all paths from source to destination
~~~~~

You are given the dimensions of a rectangular board of size  $A \times B$ . You need to print all the possible paths from top-left corner to bottom-right corner of the board. You can only move down (denoted by 'D') or right (denoted by 'R') at any point in time. You have to print the paths in lexicographical order.

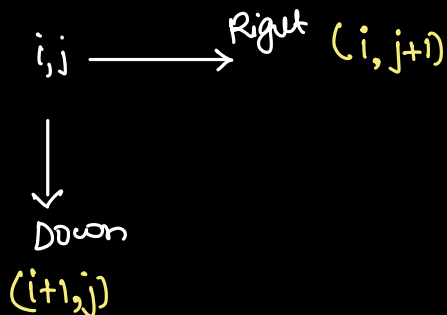
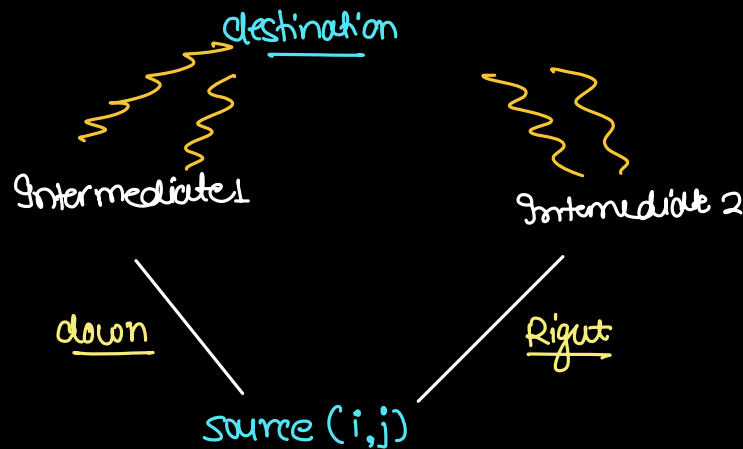
$A=3$   
 $B=2$



$A=1$   
 $B=2$



DDR, DRD, RDD



Proactive  $\rightarrow$  making smart call with normal base case

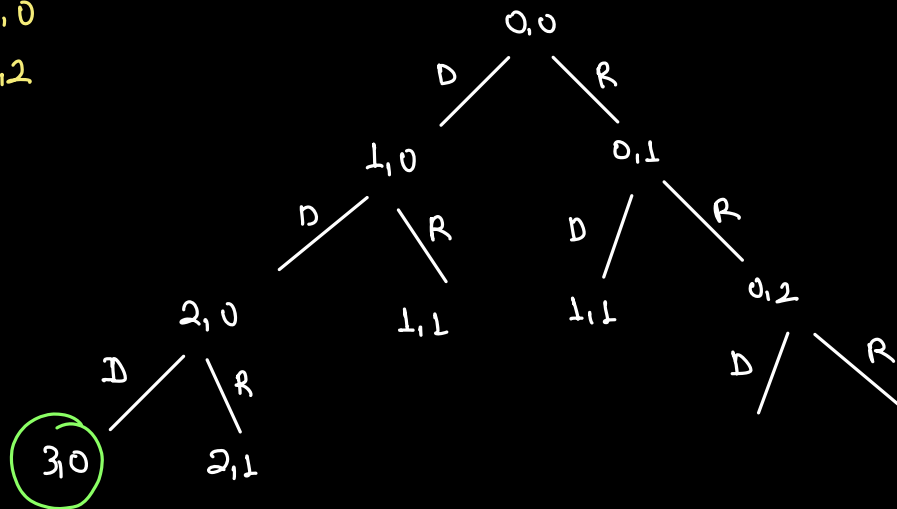
Reactive  $\rightarrow$  making foolish call with smart base case.

$A=3, B=3 \rightarrow$  print all path

source  $\rightarrow 0,0$

Dest  $\rightarrow 2,2$

Proactive  
code  
and  
Reactive



$\rightarrow$  Proactive code

will not allow this

call  $\rightarrow$  Since if intermediate is valid we will make call

Reactive code: make call & if reach at bad point, handle it in the base case.

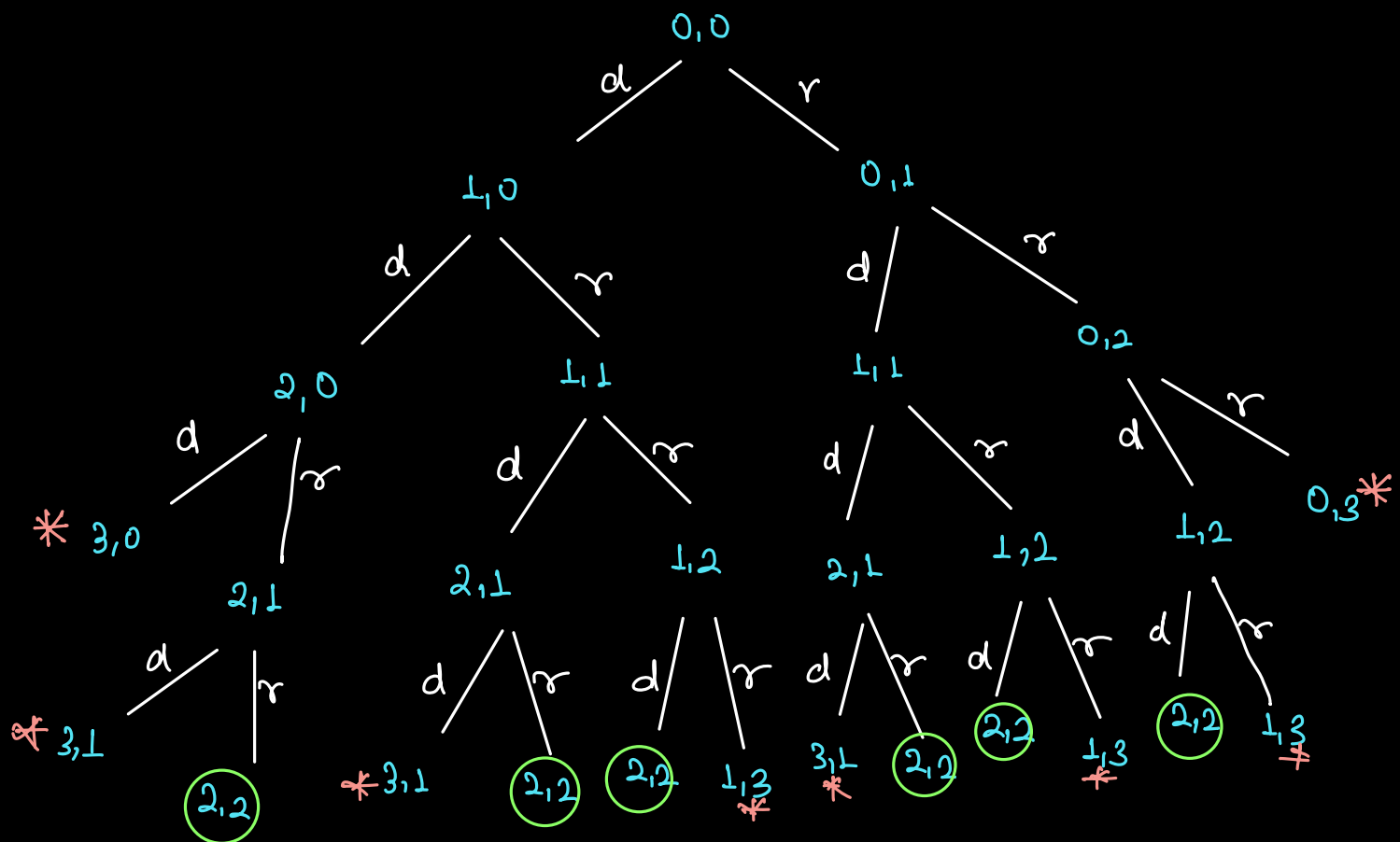
void generatePath(sr, sc, dr, dc, psf) {  
    if (sr > dr OR sc > dc) {  
        return;  
    }  
    if (sr == dr && sc == dc) {  
        ans.add(psf);  
    }  
    generatePath(sr+1, sc, dr, dc, psf+"D");  
    generatePath(sr, sc+1, dr, dc, psf+"R");  
}

$\rightarrow$  String.

T.C:  $O(2^{n+m})$

S.C:  $O(n+m)$

$$A=3, B=3$$



- ① ddr
- ② drdr
- ③ drdd
- ④ rddr
- ⑤ rdrr
- ⑥ rrrd

arranged in lexicographical order

## Shortest path in a matrix with hurdles

Given an  $M \times N$  matrix where each element can either be 0 or 1. We need to find the length of shortest path between a given source cell to a destination cell.

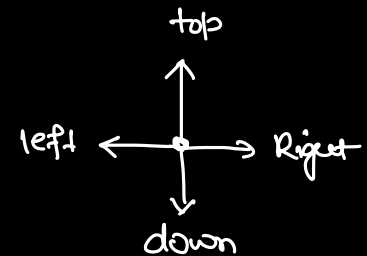
A cell with value 0 denotes that it's a hurdle. The path can only be created out of the cells with values 1.

If NO path exists then print -1.

Example 1:

A =

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | 0 |
| 2 | 0 | 0 | 1 | 1 |
| 3 | 0 | 0 | 0 | 1 |



B=0, C=0 → Src

D=3, E=3 → dst

length = 6

Example 2:

A =

|   | 0  | 1 | 2  |
|---|----|---|----|
| 0 | 1* | 1 | 1* |
| 1 | 1  | 0 | 1  |
| 2 | 1  | 1 | 1  |

B=0, C=0

D=0, E=2

path with blue → length = 2

path with orange → length = 6

min length of path = 2

Example 3:

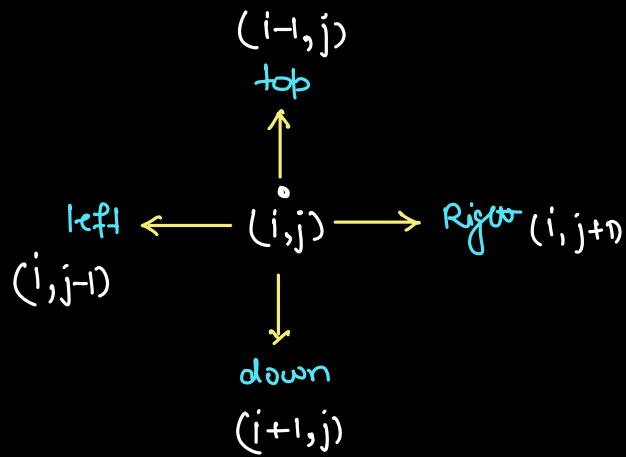
A =

|   | 0  | 1 | 2  |
|---|----|---|----|
| 0 | 1* | 0 | 1* |
| 1 | 1  | 0 | 1  |
| 2 | 1  | 0 | 1  |

B=0, C=0

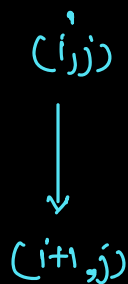
D=0, E=2

no path → -1



$x_{dir} \rightarrow \begin{matrix} T & L & D & R \\ [-1 & 0 & 1 & 0] \end{matrix}$   
 $y_{dir} \rightarrow \begin{matrix} 0 & -1 & 0 & 1 \end{matrix}$

$i, j$   
 for( $dir = 0; dir < 4; dir++$ ) {  
     int  $r = i + x_{dir}[dir]$   
     int  $c = j + y_{dir}[dir]$   
 }



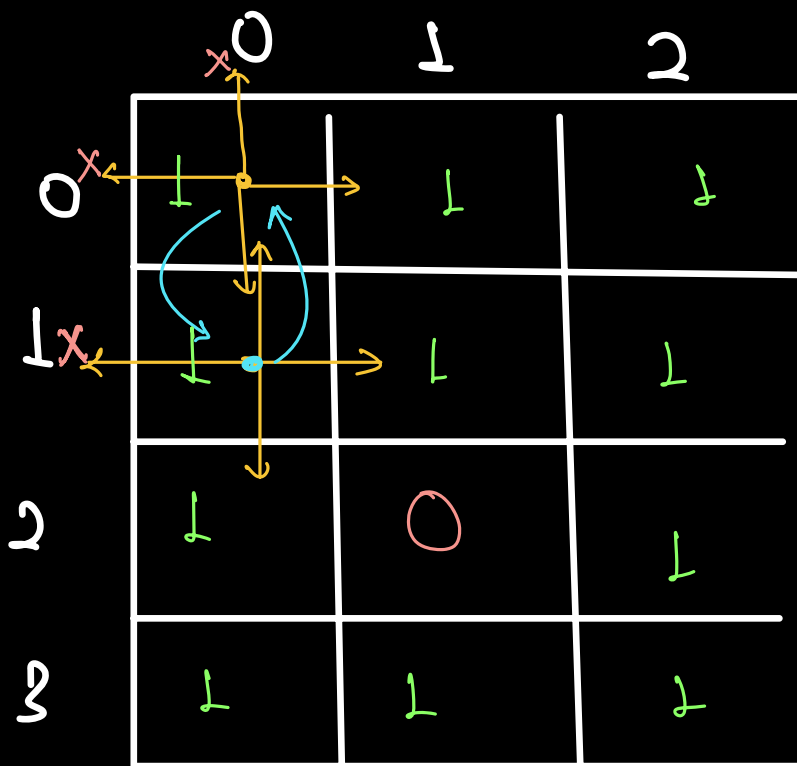
condition:

- ① Index should be valid
- ② Make sure that it is not hurdle

$\{ (i+1 < \text{total rows} \ \&\& \ \text{arr}[i+1][j] == 1) \}$

→ movement allowed.

}



To avoid infinite loop b/w 0,0 and 1,0  $\rightarrow$  we have to use visited.

$\rightarrow$  It will help us to identify cell which is already visited.

visited:  $\rightarrow$  can make another array of size  $m \times n$

My suggestion  $\rightarrow$  mark  $arr[i][j]$  by some value which can represent that it is visited

if visited  $\rightarrow$  mark it as  $arr[i][j]$  as  $-1$

$i, j \rightarrow$   $n_i$  and  $n_j$  are new row and column.

$\rightarrow$  index check AND not hurdle AND not visited.

\* mark  $n_i$  and  $n_j$  cell

\* call for that cell

\* unmark  $n_i$  and  $n_j$  cell

final answer; + xdir array + ydir array  $\rightarrow$  globally  $\rightarrow$   
int findSmallestPath(int[][] arr, int sr, int sc, int dr, int dc) {

final answer = + $\infty$

arr[sr][sc] = -1;

explorePath(arr, sr, sc, dr, dc, 0);

arr[sr][sc] = 1;

if (final ans == + $\infty$ ) return -1;

return final answer;

}

void explore(int[][] arr, int sr, sc, dr, dc, pathLength) {

if (sr == dr AND sc == dc) {

final answer = min(final ans, pathLength);

return;

}

for (int dir = 0; dir < 4; dir++) {

int ni = sr + xdir[dir];

int nj = sc + ydir[dir];

if (ni >= 0 && ni < total Rows &&

nj >= 0 && nj < total Cols &&

arr[ni][nj] == 1]  $\rightarrow$  not hurdle as well as not visited

}

arr[ni][nj] = -1;  $\rightarrow$  mark

explorePath(arr, ni, nj, dr, dc, pathLength + 1;

arr[ni][nj] = 1;  $\rightarrow$  unmark

}

}

T.C:  $O(4^{n*m})$

S.C:  $O(n*m)$

}

optional part: N queen:  $\rightarrow$  place n queens in  $n \times n$  chess board and print all possibility.

|       |       |       |       |
|-------|-------|-------|-------|
|       |       | $q_1$ |       |
| $q_2$ |       |       |       |
|       |       |       | $q_3$ |
|       | $q_4$ |       |       |

4x4

\* q \* \*

\* \* \* q

q \* \* \*

\* \* q \*

\* \* q \*

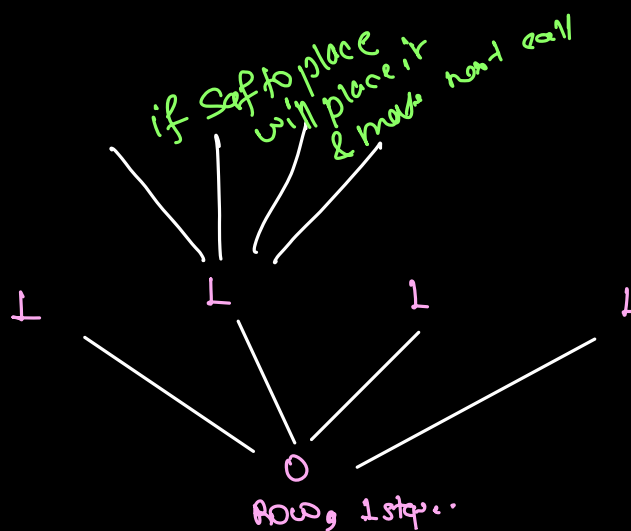
q \* \* \*

\* \* \* q

\* q \* \*

Observation:  $\rightarrow$  Each Row have one queen,  
 $\rightarrow$  if not then backtrack and fix previous mistake.

$n \times n, \rightarrow$  n queen.



All option for current que

- $\rightarrow$  check if placing a queen is safe
  - $\rightarrow$  place it
  - $\rightarrow$  call for next Row
  - $\rightarrow$  unplace it

```
void nqueen( char[][] board, int i) {
```

```
    if( i == n) {
        // all the queens are placed
        print that board
        return;
    }
```

```
    for( int j = 0; j < total colm; j++) {
```

```
        // check if (i, j) is safe to place the queen.
```

```
        if( isSafeToPlace( board, i, j) == true) {
```

```
            board[i][j] = 'q';
```

```
            nqueen( board, i+1);
```

```
            board[i][j] = '.';
```

```
        }
```

```
    }
```

```
}
```

```
boolean isSafeToPlace( char[][] board, int i, int j) {
```

```
    // TODO;
```

```
}
```

